

Question No:8

Question description

You are given a tree consisting of n nodes. A *matching* is a set of edges where each node is an endpoint of at most one edge. What is the maximum number of edges in a matching?

Input Format

The first input line contains an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.

Then there are $n-1$ lines describing the edges. Each line contains two integers a and b : there is an edge between nodes a and b .

Output Format

Print one integer: the maximum number of pairs.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Example

Input:

5

1 2

1 3

3 4

3 5

Output:

2

Explanation: One possible matching is (1,2) and (3,4)

Logical Test Cases

Test Case 1

INPUT (STDIN)

5
1 2
1 3
3 4
3 5

EXPECTED OUTPUT

2

Test Case 2

INPUT (STDIN)

5
1 1
1 2
2 3
3 4

EXPECTED OUTPUT

6

Code:

```
1  #include <stdio.h>
2
3  #define MAXN 200005
4
5  int head[MAXN];
6  int to[2 * MAXN];
7  int next[2 * MAXN];
8  int edgeCount = 0;
9
10 int parent[MAXN];
11 int order[MAXN];
12 int matched[MAXN];
13
14 void addEdge(int u, int v)
15 {
16     to[edgeCount] = v;
17     next[edgeCount] = head[u];
18     head[u] = edgeCount++;
19 }
20
21 int main()
22 {
23     int n;
24
25     scanf("%d", &n);
26
27     /* Initialize adjacency list */
28     for (int i = 1; i <= n; i++)
29         head[i] = -1;
```

```

31      /* Read n-1 edges */
32      for (int i = 0; i < n - 1; i++)
33      {
34          int a, b;
35          scanf("%d %d", &a, &b);
36
37          addEdge(a, b);
38          addEdge(b, a);
39      }
40      int count = 0;
41
42      order[count++] = 1;
43      parent[1] = 0;
44
45      for (int i = 0; i < count; i++)
46      {
47          int u = order[i];
48
49          for (int e = head[u]; e != -1; e = next[e])
50          {
51              int v = to[e];
52
53              if (v == parent[u])
54                  continue;

```

```

55
56     parent[v] = u;
57     order[count++] = v;
58 }
59 }
60
61 int answer = 0;
62
63 for (int i = n - 1; i > 0; i--)
64 {
65     int u = order[i];
66     int p = parent[u];
67
68     if (!matched[u] && !matched[p])
69     {
70         matched[u] = 1;
71         matched[p] = 1;
72
73         answer++;
74     }
75 }
76
77 printf("%d\n", answer);
78
79 return 0;
80 }

```

Output:

Test Case-1:

```

5
1 2
1 3
3 4
3 5
2

```

Test Case-2:

```
5
1 1
1 2
2 3
3 4
1
PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\DSA Practice>
```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236